Market Basket Analysis (MBA)

Analise de Cesta de Compras com Python

Autor: Renato Leone

Projeto: Analise de Dados com Linguagem Python

Objetivo

Aplicar tecnicas de Market Basket Analysis para identificar associacoes entre produtos comprados na mesn transacao e apoiar decisoes relacionadas a recomendacao de produtos, vendas cruzadas e organizacao da of

A analise utiliza regras de associacao e o algoritmo Apriori, avaliando as metricas Support, Confidence e Li

Abordagem

O projeto contempla:

1. Carregamento e inspecao dos dados;
2. Analise exploratoria;
3. Verificacao de valores ausentes;
4. Integracao das tabelas;
5. Preparacao das transacoes;
6. Analise do comportamento de compra e recompra;
7. Aplicacao do algoritmo Apriori;
8. Interpretacao das regras de associacao.

> Observacao: o notebook utiliza o conjunto de dados Instacart Market Basket Analysis. Os dados brutos nao fazem parte deste notebook e devem ser disponibilizados na pasta `dados/` para execucao completa.

Ambiente e Dependencias

Versao de referencia do Python: `3.11.3`

Principais bibliotecas utilizadas:

- pandas `1.5.3`
- numpy `1.24.3`
- matplotlib `3.7.1`

- efficient-apriori `2.0.3`
- watermark

Para o GitHub, as versoes tambem sao registradas no arquivo `requirements.txt`.

In [ ]:
```
# Versao da Linguagem Python
from platform import python_version
print('Versao da Linguagem Python Usada Neste Jupyter Notebook:', python_
```

```
Versao da Linguagem Python Usada Neste Jupyter Notebook: 3.11.3
```

In [2]:
```
from IPython.display import display, HTML
display(HTML("<style>.container{width: 80% !important;}</style>"))
```

In [4]:
```
# Instale o pacote apenas se ele ainda nao estiver disponivel no ambiente
# !pip install -q efficient_apriori
```

In [5]:
```
# Imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime
from itertools import combinations
import efficient_apriori
from efficient_apriori import apriori
```

In [ ]:
```
# Versoes dos pacotes usados neste jupyter notebook
%reload_ext watermark
%watermark -a "Renato Leone" --iversions
```

```
Author: Renato Leone

pandas           : 1.5.3
matplotlib       : 3.7.1
efficient_apriori: 2.0.3
numpy            : 1.24.3
```

Carregamento dos Dados

O projeto utiliza seis arquivos CSV, armazenados na pasta `dados/`:

Arquivo	Conteudo
`aisles.csv`	Corredores/categorias de produtos
`departments.csv`	Departamentos
`orders.csv`	Pedidos e informacoes da compra
`products.csv`	Cadastro de produtos
`order_products__prior.csv`	Produtos dos pedidos anteriores
`order_products__train.csv`	Produtos dos pedidos do conjunto de treino

O arquivo `order_products__prior.csv` e utilizado como principal base transacional para a cons das cestas e aplicacao do Apriori.

In [7]:
```
# Carrega os dados
aisle = pd.read_csv('dados/aisles.csv')
department = pd.read_csv('dados/departments.csv')
order = pd.read_csv('dados/orders.csv')
product = pd.read_csv('dados/products.csv')
order_product_prior = pd.read_csv('dados/order_products__prior.csv')
order_product_train = pd.read_csv('dados/order_products__train.csv')
```

Analise Exploratoria

Observacao sobre correlacao

A correlacao e uma medida voltada principalmente para avaliar associacao entre variaveis quantitativas. Ne projeto, grande parte das colunas de identificacao (`*_id`) representa apenas chaves de relacionamento.

Por isso, calcular correlacao diretamente sobre IDs nao produz um insight de negocio relevante. A analise concentrada nas variaveis que representam comportamento de compra, produtos, departamentos, horarios recompra.

In [8]:
```
aisle.shape
```

Out[8]:
```
(134, 2)
```

In [20]:
```
aisle
```

Out[20]:

	aisle_id	aisle
0	1	prepared soups salads
1	2	specialty cheeses
2	3	energy granola bars
3	4	instant foods
4	5	marinades meat preparation
...	...	...
129	130	hot cereal pancake mixes
130	131	dry pasta
131	132	beauty
132	133	muscles joints pain relief
133	134	specialty wines champagnes

134 rows x 2 columns

In [10]:
```
department.shape
```

Out[10]:
```
(21, 2)
```

In [11]:
```
department.head()
```

Out[11]:

	department_id	department
0	1	frozen
1	2	other
2	3	bakery
3	4	produce
4	5	alcohol

In [12]:
```
product.shape
```

Out[12]:
```
(49688, 4)
```

In [13]:
```
product.head()
```

Out[13]:

	product_id	product_name	aisle_id	department_id
0	1	Chocolate Sandwich Cookies	61	19
1	2	All-Seasons Salt	104	13
2	3	Robust Golden Unsweetened Oolong Tea	94	7
3	4	Smart Ones Classic Favorites Mini Rigatoni Wit...	38	1
4	5	Green Chile Anytime Sauce	5	13

In [14]:
```
order.shape
```

Out[14]: (3421083, 7)

In [15]:
```
order.head()
```

Out[15]:

	order_id	user_id	eval_set	order_number	order_dow	order_hour_of_day	days_since_
0	2539329	1	prior	1	2	8	
1	2398795	1	prior	2	3	7	
2	473747	1	prior	3	3	12	
3	2254736	1	prior	4	4	7	
4	431534	1	prior	5	4	15	

In [16]:
```
order_product_prior.shape
```

Out[16]: (32434489, 4)

In [17]:
```
order_product_prior.head()
```

Out[17]:

	order_id	product_id	add_to_cart_order	reordered
0	2	33120	1	1
1	2	28985	2	1
2	2	9327	3	0
3	2	45918	4	1
4	2	30035	5	0

In [18]:
```
order_product_train.shape
```

Out[18]: (1384617, 4)

In [19]:
```
order_product_train.head()
```

Out[19]:

	order_id	product_id	add_to_cart_order	reordered
0	1	49302	1	1
1	1	11109	2	1
2	1	10246	3	0
3	1	49683	4	0
4	1	43633	5	1

Validacao do volume de dados

Antes das analises, e importante confirmar programaticamente a quantidade de registros de cada dataset. Is
permite validar a carga e detectar rapidamente arquivos incompletos ou inconsistentes.

In [21]:
```
# Total de registros por dataset
count_array = np.array([['Aisles', len(aisle)],
                        ['Department', len(department)],
                        ['Product', len(product)],
                        ['Order', len(order)],
                        ['Order_Product_Prior', len(order_product_prior)]
                        ['Order_Product_Train', len(order_product_train)]
count_record = pd.DataFrame(count_array, columns = ['Arquivo', 'Total de
count_record
```

Out[21]:

	Arquivo	Total de Registros
0	Aisles	134
1	Department	21
2	Product	49688
3	Order	3421083
4	Order_Product_Prior	32434489
5	Order_Product_Train	1384617

In [22]:
```
# Total de itens nas categorias de eval_set
order['eval_set'].value_counts()
```

Out[22]:
```
prior    3214874
train     131209
test       75000
Name: eval_set, dtype: int64
```

Qualidade dos Dados

Nesta etapa verificamos valores ausentes e avaliamos se eles exigem tratamento antes da analise.

Verificando Valores Ausentes

In [23]:
```
aisle.isna().sum()
```

Out[23]:
```
aisle_id    0
aisle       0
dtype: int64
```

In [24]:
```
department.isna().sum()
```

Out[24]:
```
department_id    0
department       0
dtype: int64
```

In [25]:
```
order.isna().sum()
```

Out[25]:
```
order_id                      0
user_id                       0
eval_set                      0
order_number                  0
order_dow                     0
order_hour_of_day             0
days_since_prior_order    206209
dtype: int64
```

In [26]:
```
product.isna().sum()
```

Out[26]:
```
product_id       0
product_name     0
aisle_id         0
department_id    0
dtype: int64
```

In [27]:
```
order_product_prior.isna().sum()
```

Out[27]:
```
order_id             0
product_id           0
add_to_cart_order    0
reordered            0
dtype: int64
```

In [28]:
```
order_product_train.isna().sum()
```

Out[28]:
```
order_id             0
product_id           0
add_to_cart_order    0
reordered            0
dtype: int64
```

Preparacao e Integracao dos Dados

Nesta etapa, as tabelas transacionais sao enriquecidas com informacoes de pedidos, produtos, corredores e departamentos.

Merge da Tabela order_product_prior com order, product, aisle e departm

In [29]:
```
# Merge
order_product_prior = order_product_prior.merge(order, on = 'order_id', h
```

```
order_product_prior = order_product_prior.merge(product, on = 'product_id
order_product_prior = order_product_prior.merge(aisle, on = 'aisle_id', h
order_product_prior = order_product_prior.merge(department, on = 'departm
```

In [30]:
```
order_product_prior.head(10)
```

Out[30]:

	order_id	product_id	add_to_cart_order	reordered	user_id	eval_set	order_number
0	2	33120	1	1	202279	prior	3
1	2	28985	2	1	202279	prior	3
2	2	9327	3	0	202279	prior	3
3	2	45918	4	1	202279	prior	3
4	2	30035	5	0	202279	prior	3
5	2	17794	6	1	202279	prior	3
6	2	40141	7	1	202279	prior	3
7	2	1819	8	1	202279	prior	3
8	2	43668	9	0	202279	prior	3
9	3	33754	1	1	205970	prior	16

In [31]:
```
order_product_prior.isna().sum()
```

Out[31]:
```
order_id                      0
product_id                    0
add_to_cart_order             0
reordered                     0
user_id                       0
eval_set                      0
order_number                  0
order_dow                     0
order_hour_of_day             0
days_since_prior_order    2078068
product_name                  0
aisle_id                      0
department_id                 0
aisle                         0
department                    0
dtype: int64
```

In [32]:
```
order_product_prior.isnull().sum() / len(order_product_prior) * 100
```

Out[32]:
```
order_id                  0.00000
product_id                0.00000
add_to_cart_order         0.00000
reordered                 0.00000
user_id                   0.00000
eval_set                  0.00000
order_number              0.00000
order_dow                 0.00000
order_hour_of_day         0.00000
days_since_prior_order    6.40697
product_name              0.00000
aisle_id                  0.00000
department_id             0.00000
aisle                     0.00000
department                0.00000
dtype: float64
```

Tratamento de `days_since_prior_order`

A variavel `days_since_prior_order` apresenta aproximadamente $6,4$ % de valores ausentes no analisado.

Neste projeto, a variavel sera mantida porque:

1. o percentual de ausencia e relativamente pequeno;
2. ela nao e utilizada como entrada no algoritmo de Market Basket Analysis.

Portanto, nao sera feita imputacao dessa variavel.

Merge da Tabela order_product_train com order, product, aisle e departm

In [33]:
```
%%time
order_product_train = order_product_train.merge(order, on = 'order_id', h
order_product_train = order_product_train.merge(product, on = 'product_id
```

```
order_product_train = order_product_train.merge(aisle, on = 'aisle_id', h
order_product_train = order_product_train.merge(department, on = 'departm
```

```
CPU times: user 591 ms, sys: 294 ms, total: 885 ms
Wall time: 912 ms
```

In [34]:
```
order_product_train.head(10)
```

Out[34]:

	order_id	product_id	add_to_cart_order	reordered	user_id	eval_set	order_number
0	1	49302	1	1	112108	train	4
1	1	11109	2	1	112108	train	4
2	1	10246	3	0	112108	train	4
3	1	49683	4	0	112108	train	4
4	1	43633	5	1	112108	train	4
5	1	13176	6	0	112108	train	4
6	1	47209	7	0	112108	train	4
7	1	22035	8	1	112108	train	4
8	36	39612	1	0	79431	train	23
9	36	19660	2	1	79431	train	23

In [35]:
```
order_product_train.isna().sum()
```

Out[35]:
```
order_id                  0
product_id                0
add_to_cart_order         0
reordered                 0
user_id                   0
eval_set                  0
order_number              0
order_dow                 0
order_hour_of_day         0
days_since_prior_order    0
product_name              0
aisle_id                  0
department_id             0
aisle                     0
department                0
dtype: int64
```

Preparacao das transacoes para o Apriori

Vamos agrupar os produtos por `order_id` para transformar os registros de itens em cestas de produtos pedido passa a representar uma transacao contendo a lista de produtos comprados.

Essa estrutura sera utilizada posteriormente pelo algoritmo Apriori.

In [36]:
```
%%time
order_grouped = pd.DataFrame(order_product_prior.groupby('order_id')['pro
```

```
CPU times: user 31.7 s, sys: 24.8 s, total: 56.5 s
Wall time: 1min 8s
```

In [37]:
```
order_grouped.shape
```

Out[37]:
```
(3214874, 2)
```

In [38]:
```
len(order_grouped[0])
```

Out[38]:
```
3214874
```

In [39]:
```
order_grouped.head()
```

Out[39]:

	0
0	2 0 33120 1 28985 2 9327 3 45918 4
1	3 9 33754 10 24838 11 17704 12 219
2	4 17 46842 18 26434 19 39758 20 277
3	5 30 13176 31 15005 32 47329 33 279
4	6 56 40462 57 15873 58 41897 Name: p

In [40]:
```
# Listas
prod = []
```

In [41]:
```

```

```
%%time
for i in range(len(order_grouped[0])):
    prod.append(list(order_grouped.iloc[i][1]))
```

```
CPU times: user 1min 4s, sys: 9.47 s, total: 1min 13s
Wall time: 1min 18s
```

In [42]:
```
prod_ = prod
```

In [43]:
```
order_ = order_grouped[0]
```

In [44]:
```
# Prepara o dataframe
transactions = pd.DataFrame({'OrderID' : order_, 'Products' : prod_})
```

In [45]:
```
transactions.head(10)
```

Out[45]:

	OrderID	
0	2	[3 3 1 2 0, 2 8 9 8 5, 9 3 2 7, 4 5 9 1 8, 3 0 1 7 7 9 4, 4 0
1	3	[3 3 7 5 4, 2 4 8 3 8, 1 7 7 0 4, 2 1 9 0 3, 1 7 4 6 6 6 7, 1
2	4	[4 6 8 4 2, 2 6 4 3 4, 3 9 7 5 8, 2 7 7 6 1, 1 0 2 1 3 5 1, 2
3	5	[1 3 1 7 6, 1 5 0 0 5, 4 7 3 2 9, 2 7 9 6 6, 2 3 4 8 3 7 0, 1
4	6	[4 0 4 6 2, 1 5 8 7 3, 4 1
5	7	[3 4 0 5 0, 4 6
6	8	[2 3
7	9	[2 1 4 0 5, 4 7 8 9 0, 1 1 1 8 2, 2 0 1 4, 2 9 3 4 2 0 3, 1 4
8	1 0	[2 4 8 5 2, 4 7 9 6, 3 1 7 1 7, 4 7 7 6 6, 4 6 0 5, 1 2 1 1
9	1 1	[3 0 1 6 2, 2 7 0 8 5, 5 9 9 4, 1 3 1 3, 3 1

Analise Descritiva do Comportamento de Compra

Antes da aplicacao do Apriori, analisamos padroes de pedidos, horarios, departamentos, corredores, produt comportamento de recompra.

Criterio de leitura: nas analises de volume, uma linha da tabela `order_product_prior` representa u associado a um pedido. Portanto, as contagens de produtos, corredores e departamentos representam ocor itens, e nao quantidade distinta de pedidos.

1 . Qual e o numero de pedidos mais frequente entre os usuarios?

In [46]:
```
# Agrupa os dados por usuario e numero do pedido, agregando pelo maior va
ord_max_count = order.groupby("user_id")['order_number'].aggregate(np.max
ord_max_count.head()
```

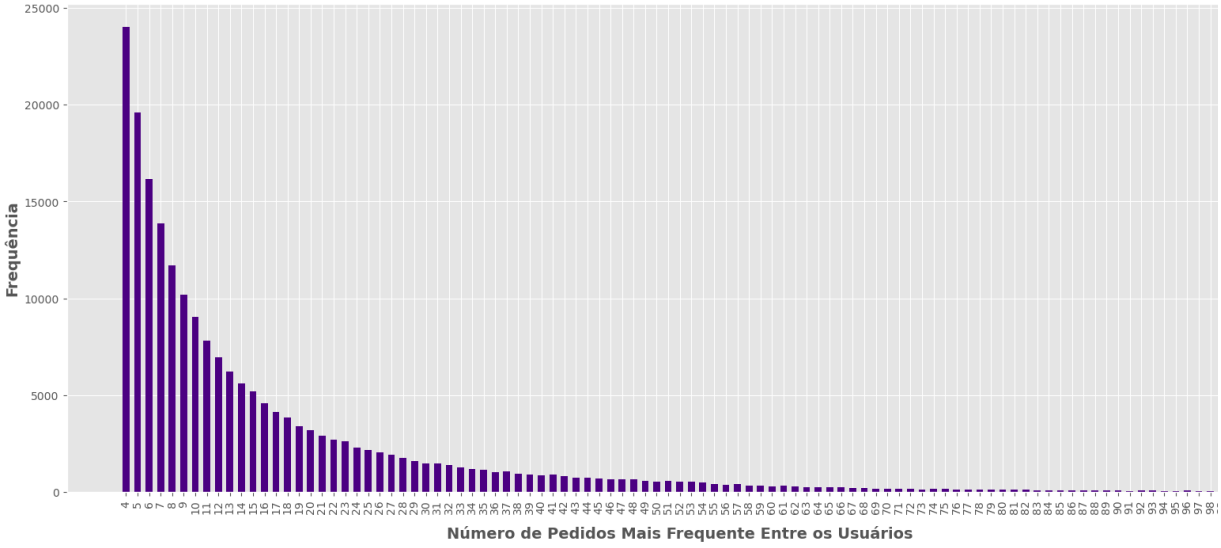
Out[46]:

	user_id	order_number
0	1	1 1
1	2	1 5
2	3	1 3
3	4	6
4	5	5

In [47]:
```
# Faz a contagem por numero de pedidos
ord_max_count = ord_max_count.order_number.value_counts()
ord_max_count.head()
```

Out[47]:
```
4    23986
5    19590
6    16165
7    13850
8    11700
Name: order_number, dtype: int64
```

In [48]:
```
# Distribuicao do numero maximo de pedidos realizados por usuario
plt.figure(figsize=(14, 6))
plt.bar(ord_max_count.index, ord_max_count.values, width=0.6)
plt.title('Distribuicao do Numero de Pedidos por Usuario', fontsize=15, f
plt.xlabel('Numero de Pedidos Realizados', fontsize=12)
plt.ylabel('Quantidade de Usuarios', fontsize=12)
plt.xticks(ord_max_count.index, rotation=90)
plt.tight_layout()
plt.show()
```

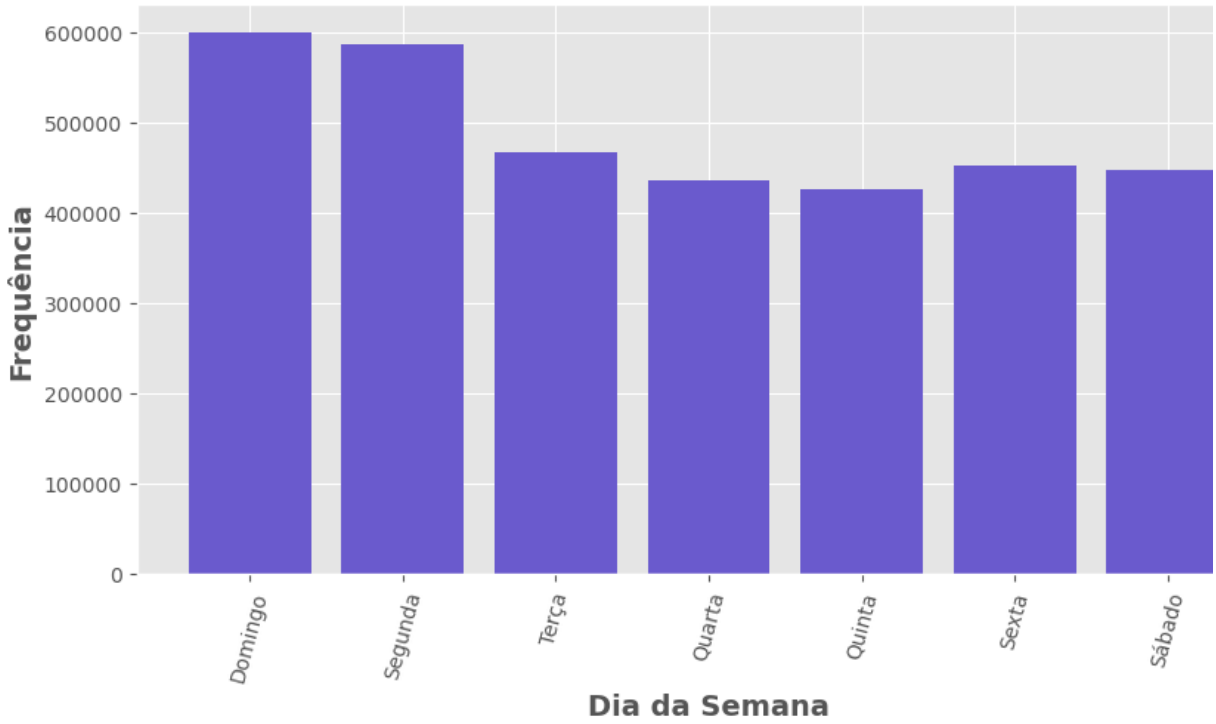


2 . Qual dia da semana apresenta o maior numero de pedidos?

In [92]:
```
# Frequencias de pedidos por dia da semana (dow = day of the week)
x = [0,1,2,3,4,5,6]
y = [order[order['order_dow']==0].shape[0],
     order[order['order_dow']==1].shape[0],
     order[order['order_dow']==2].shape[0],
     order[order['order_dow']==3].shape[0],
     order[order['order_dow']==4].shape[0],
     order[order['order_dow']==5].shape[0],
     order[order['order_dow']==6].shape[0]]
```

In [94]:
```
# Frequencia de pedidos por dia da semana
x = [0, 1, 2, 3, 4, 5, 6]
y = [order[order['order_dow'] == i].shape[0] for i in x]

plt.figure(figsize=(10, 5))
plt.bar(x, y)
plt.title('Pedidos por Dia da Semana', fontsize=15, fontweight='bold')
plt.xlabel('Dia da Semana', fontsize=12)
plt.ylabel('Quantidade de Pedidos', fontsize=12)
plt.xticks(x, ['Domingo', 'Segunda', 'Terca', 'Quarta', 'Quinta', 'Sexta'
plt.tight_layout()
plt.show()
```

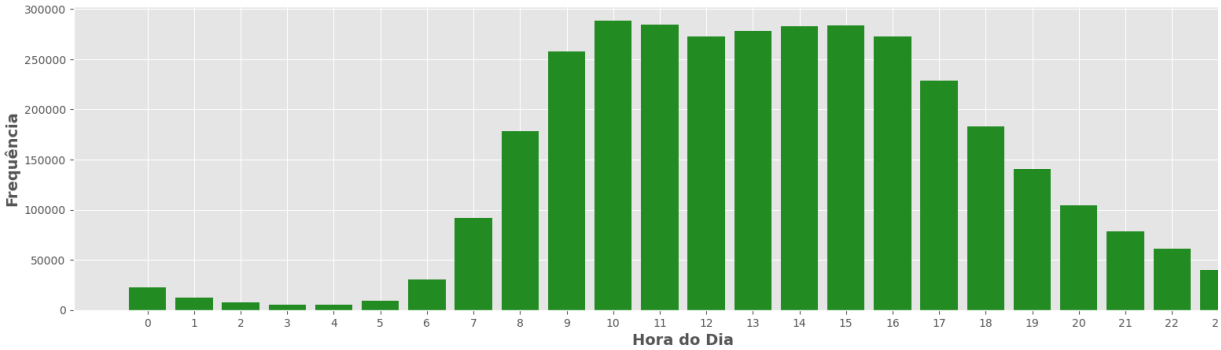


3 . Qual horario do dia apresenta o maior numero de pedidos?

In [51]:
```
# Frequencias de pedidos pela hora do dia
x = list(range(0,24))
y = []
for i in range(24):
    y.append(order[order['order_hour_of_day']==i].shape[0])
```

In [52]:
```
# Frequencia de pedidos por hora do dia
x = list(range(24))
y = [order[order['order_hour_of_day'] == i].shape[0] for i in x]
```

```
plt.figure(figsize=(12, 5))
plt.bar(x, y)
plt.title('Pedidos por Hora do Dia', fontsize=15, fontweight='bold')
plt.xlabel('Hora do Dia', fontsize=12)
plt.ylabel('Quantidade de Pedidos', fontsize=12)
plt.xticks(np.arange(0, 24, 1))
plt.tight_layout()
plt.show()
```

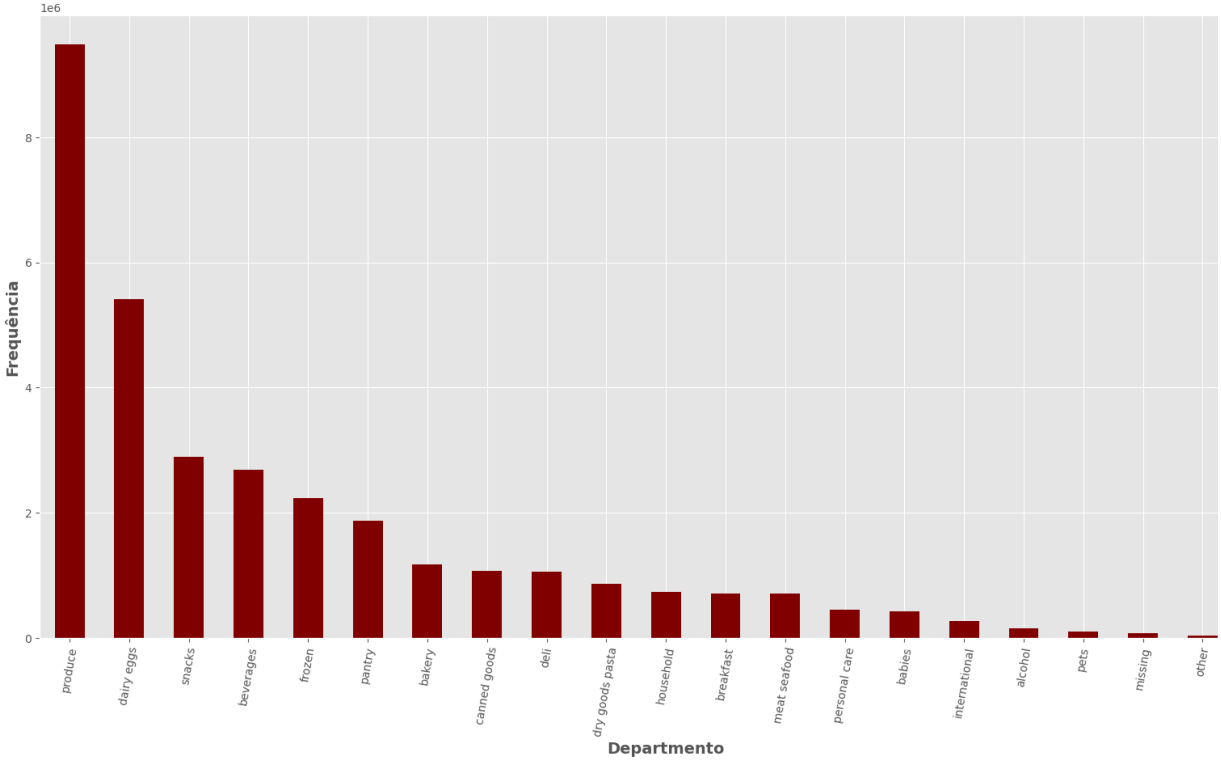


4 . Qual departamento apresenta maior volume de itens nos pedidos?

In [53]:
```
# Contagem de pedidos por departamento
dept_cnt = order_product_prior['department'].value_counts()
```

In [89]:
```
# Contagem de itens por departamento
dept_cnt = order_product_prior['department'].value_counts().sort_values()

plt.figure(figsize=(10, 8))
plt.barh(dept_cnt.index, dept_cnt.values)
plt.title('Volume de Itens por Departamento', fontsize=15, fontweight='bo
plt.xlabel('Quantidade de Itens', fontsize=12)
plt.ylabel('Departamento', fontsize=12)
plt.tight_layout()
plt.show()
```

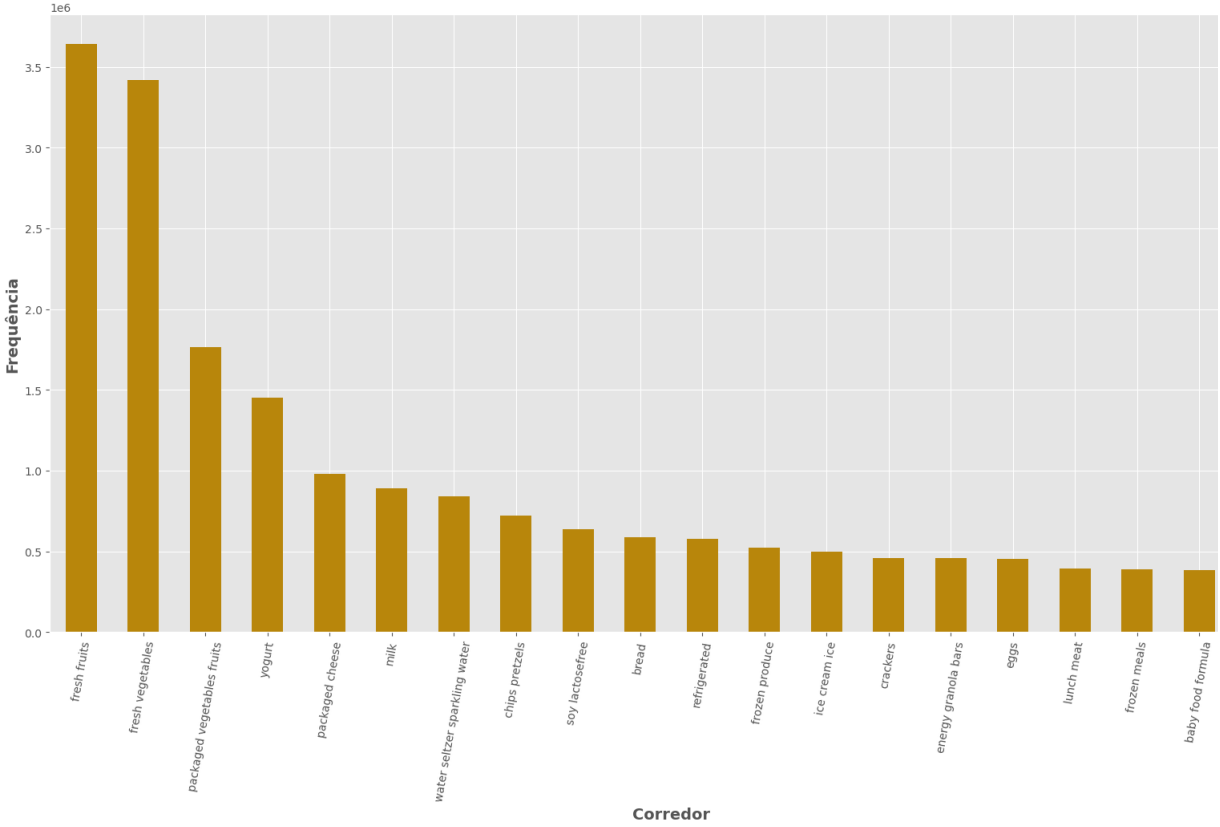


5. Quais sao os 20 principais corredores por frequencia de itens pedidos?

In [55]:
```
# Os 20 principais corredores e sua frequencia de pedidos
aisle_cnt = order_product_prior['aisle'].value_counts()
```

In [88]:
```
# Os 20 principais corredores por frequencia de itens
aisle_cnt = order_product_prior['aisle'].value_counts().head(20).sort_val

plt.figure(figsize=(10, 8))
plt.barh(aisle_cnt.index, aisle_cnt.values)
plt.title('Top 20 Corredores por Volume de Itens', fontsize=15, fontweigh
plt.xlabel('Quantidade de Itens', fontsize=12)
plt.ylabel('Corredor', fontsize=12)
plt.tight_layout()
plt.show()
```

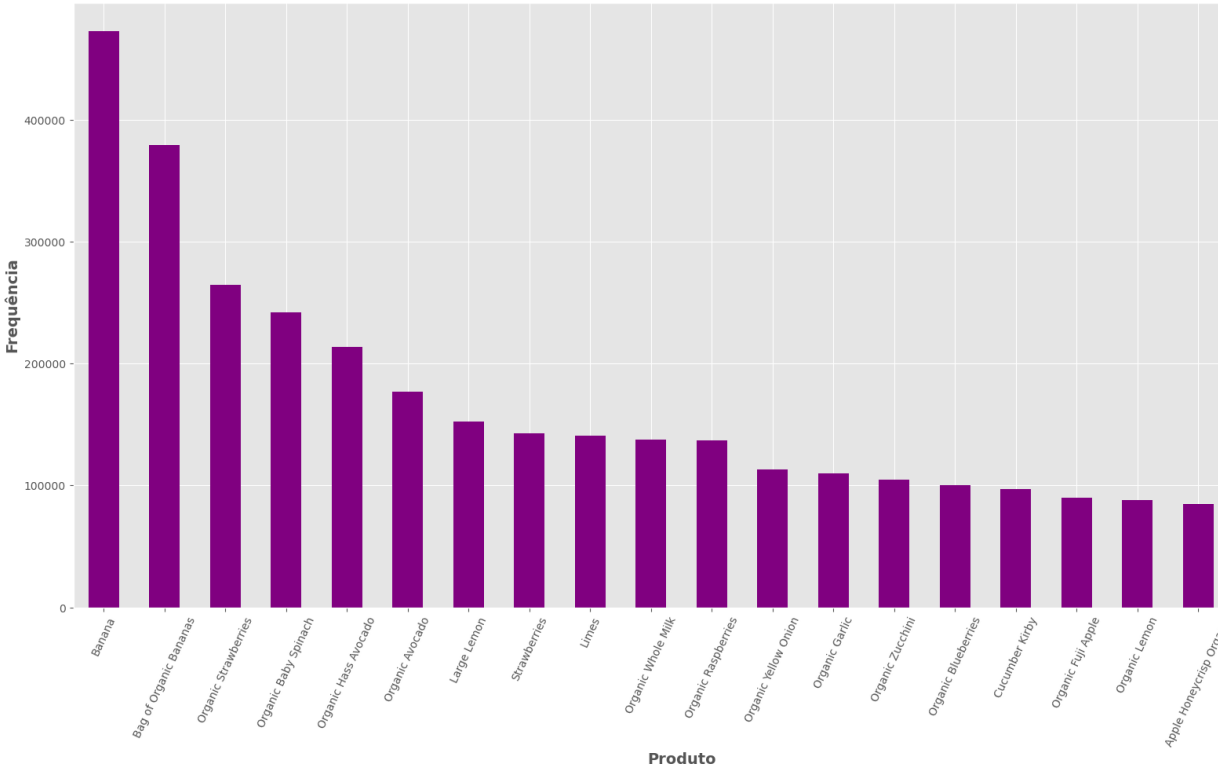


6. Quais sao os 20 principais produtos por frequencia nos pedi

In [57]:
```
# Os 20 principais produtos por frequencia de pedido
prod_cnt = order_product_prior['product_name'].value_counts()
```

In [85]:
```
# Os 20 principais produtos por frequencia
prod_cnt = order_product_prior['product_name'].value_counts().head(20).so

plt.figure(figsize=(11, 9))
plt.barh(prod_cnt.index, prod_cnt.values)
plt.title('Top 20 Produtos por Frequencia nos Pedidos', fontsize=15, font
plt.xlabel('Quantidade de Itens', fontsize=12)
plt.ylabel('Produto', fontsize=12)
plt.tight_layout()
plt.show()
```



7 . Quais produtos apresentam maior volume de recompra?

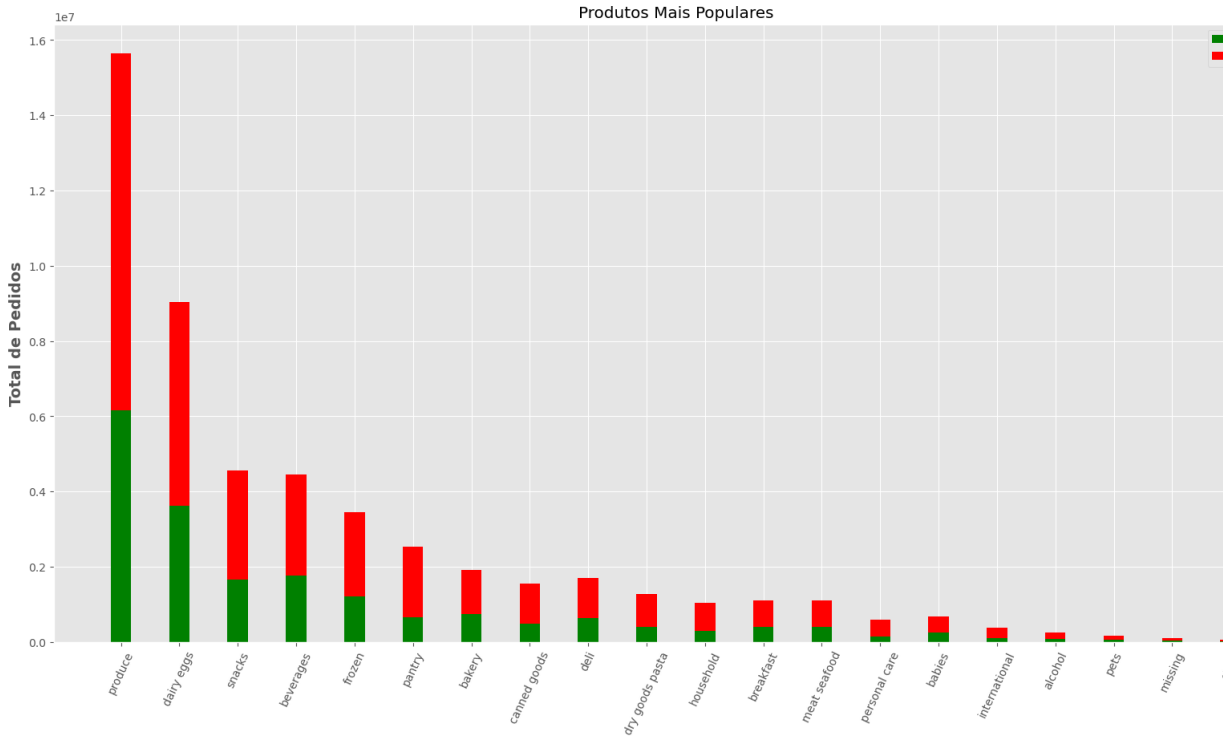
In [61]:
```
%%time
temp_df = order_product_prior.groupby("product_name")["reordered"].agg(['
temp_df = temp_df.sort_values('total', ascending = False).reset_index()
```

```
CPU times: user 2.08 s, sys: 253 ms, total: 2.33 s
Wall time: 2.5 s
```

In [62]:
```
# Prepara as listas
labels = list(temp_df.product_name[0:20])
reorder = list(temp_df.reorders[0:20])
total = list(temp_df.total[0:20])
```

In [84]:
```
# Produtos mais populares: total de ocorrencias x reorders
temp_df = (
    order_product_prior.groupby('product_name')['reordered']
    .agg(total='count', reorders='sum')
    .sort_values('total', ascending=False)
    .head(20)
    .sort_values('total')
)

plt.figure(figsize=(11, 9))
plt.barh(temp_df.index, temp_df['total'], label='Total de ocorrencias')
plt.barh(temp_df.index, temp_df['reorders'], label='Reorders')
plt.title('Top 20 Produtos: Ocorrencias e Recompras', fontsize=15, fontwe
plt.xlabel('Quantidade de Ocorrencias', fontsize=12)
plt.ylabel('Produto', fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()
```



8. Qual e a taxa media de recompra por departamento?

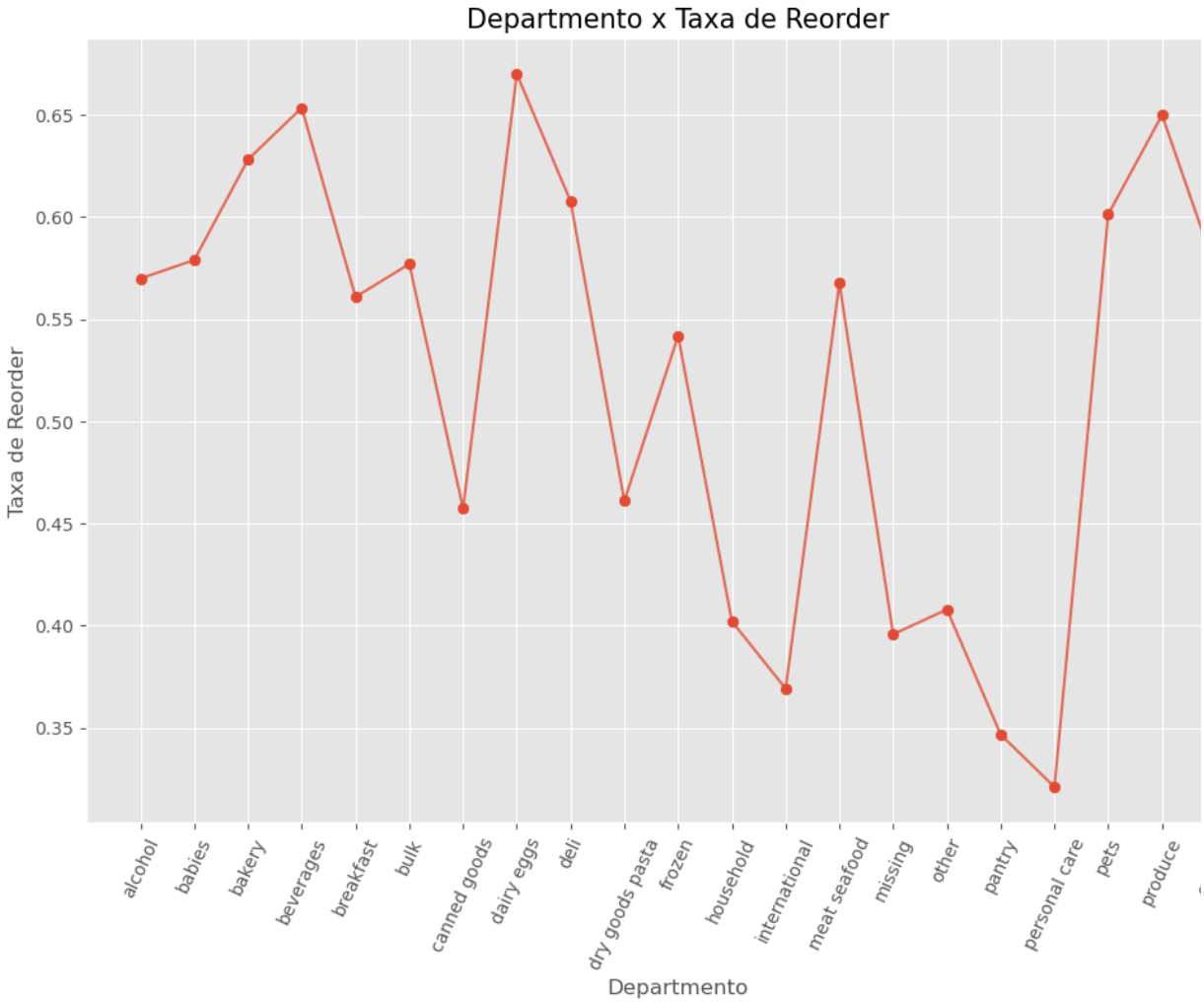
In [81]:
```
# Agrupando os dados por departamento e reorder
grouped_df = order_product_prior.groupby(["department"])["reordered"].agg
grouped_df.head()
```

Out[81]:

	department	reordered
0	alcohol	0.569924
1	babies	0.578971
2	bakery	0.628141
3	beverages	0.653460
4	breakfast	0.560922

In [83]:
```
# Taxa media de recompra por departamento
grouped_df = (
    order_product_prior.groupby('department')['reordered']
    .mean()
    .sort_values()
    .reset_index()
)

plt.figure(figsize=(10, 8))
plt.barh(grouped_df['department'], grouped_df['reordered'])
plt.title('Taxa Media de Recompra por Departamento', fontsize=15, fontwei
plt.xlabel('Taxa de Recompra', fontsize=12)
plt.ylabel('Departamento', fontsize=12)
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```



9 . Qual e o volume de pedidos e reorders por departamento?

In [66]:
```
%%time
temp_df = order_product_prior.groupby("department")["reordered"].agg(['co
temp_df = temp_df.sort_values('total', ascending = False).reset_index()
```

```
CPU times: user 1.22 s, sys: 242 ms, total: 1.46 s
Wall time: 1.57 s
```

In [67]:
```
temp_df.head()
```

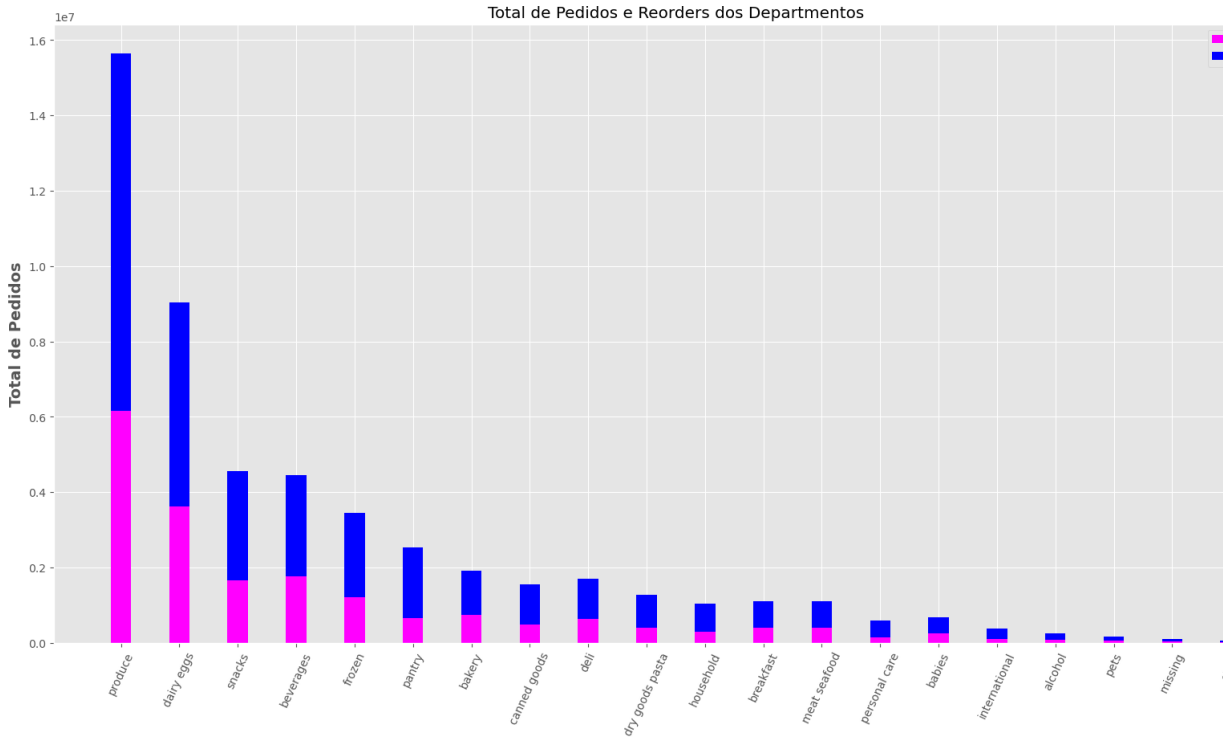
Out[67]:

	department	total	reorders
0	produce	9479291	6160710
1	dairy eggs	5414016	3627221
2	snacks	2887550	1657973
3	beverages	2690129	1757892
4	frozen	2236432	1211890

In [68]:
```
# Listas
labels = list(temp_df.department[0:20])
reorder = list(temp_df.reorders[0:20])
total = list(temp_df.total[0:20])
```

In [77]:
```
# Volume de ocorrencias e reorders por departamento
temp_df = (
    order_product_prior.groupby('department')['reordered']
    .agg(total='count', reorders='sum')
    .sort_values('total', ascending=False)
    .head(20)
    .sort_values('total')
)

plt.figure(figsize=(10, 8))
plt.barh(temp_df.index, temp_df['total'], label='Total de ocorrencias')
plt.barh(temp_df.index, temp_df['reorders'], label='Reorders')
plt.title('Departamentos: Ocorrencias e Recompras', fontsize=15, fontweig
plt.xlabel('Quantidade de Ocorrencias', fontsize=12)
plt.ylabel('Departamento', fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()
```



10. Qual e a taxa de recompra por corredor?

In [98]:
```
grouped_df = order_product_prior.groupby(["aisle"])["reordered"].aggregat
grouped_df.head()
```

Out[98]:

	aisle	reordered
0	air fresheners candles	0.309052
1	asian foods	0.351132
2	baby accessories	0.550980
3	baby bath body care	0.295770
4	baby food formula	0.590259

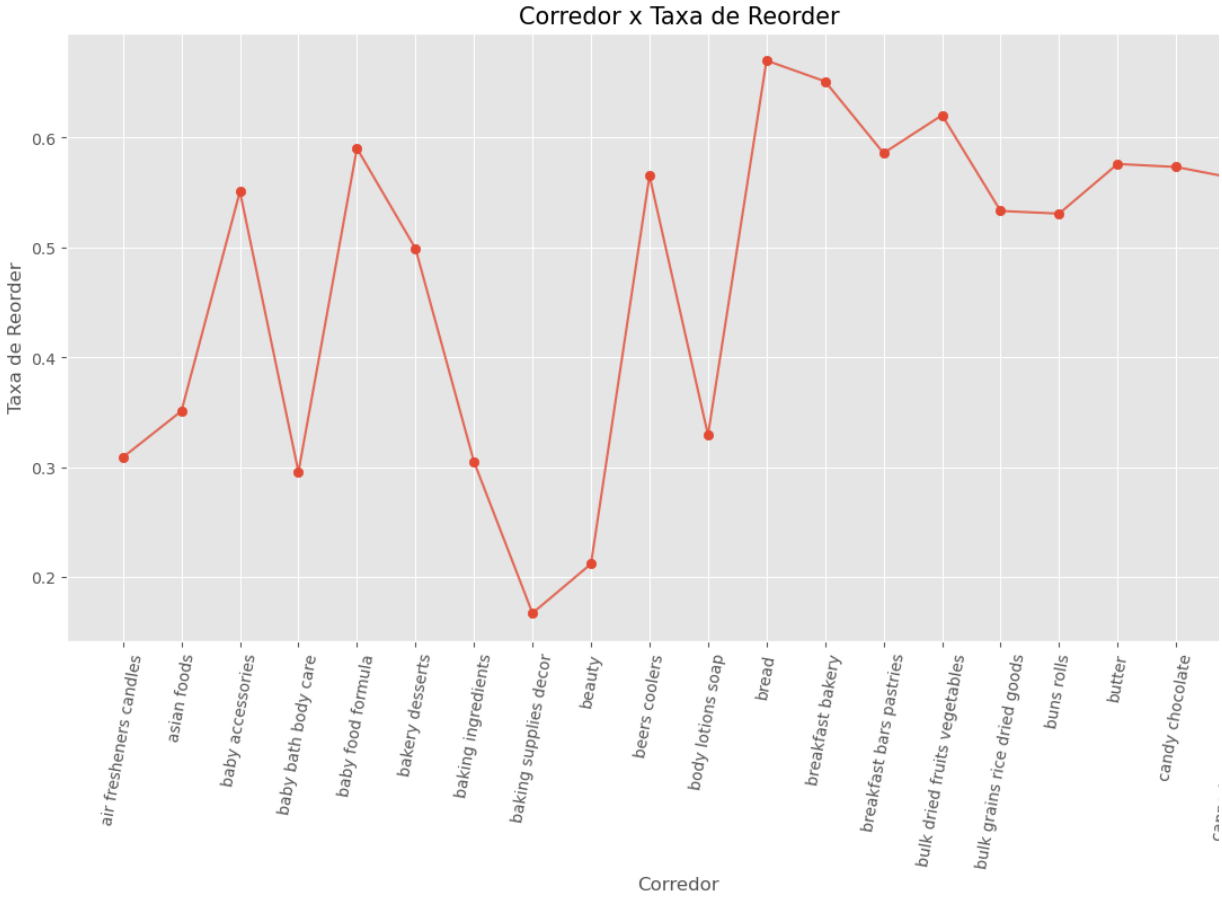
In [99]:
```
list(grouped_df['aisle'])[0:20]
```

Out[99]:
```
['air fresheners candles',
 'asian foods',
 'baby accessories',
 'baby bath body care',
 'baby food formula',
 'bakery desserts',
 'baking ingredients',
 'baking supplies decor',
 'beauty',
 'beers coolers',
 'body lotions soap',
 'bread',
 'breakfast bakery',
 'breakfast bars pastries',
 'bulk dried fruits vegetables',
 'bulk grains rice dried goods',
 'buns rolls',
 'butter',
 'candy chocolate',
 'canned fruit applesauce']
```

In [101…
```
# Taxa de recompra por corredor - foco nos 20 corredores de maior volume
aisle_volume = order_product_prior['aisle'].value_counts().head(20).index

grouped_df = (
    order_product_prior[order_product_prior['aisle'].isin(aisle_volume)]
    .groupby('aisle')['reordered']
    .mean()
    .sort_values()
    .reset_index()
)

plt.figure(figsize=(10, 8))
plt.barh(grouped_df['aisle'], grouped_df['reordered'])
plt.title('Taxa de Recompra nos 20 Principais Corredores', fontsize=15, f
plt.xlabel('Taxa de Recompra', fontsize=12)
plt.ylabel('Corredor', fontsize=12)
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```



11. Qual e o volume de pedidos e reorders por corredor?

In [102…
```
%%time
temp_df = order_product_prior.groupby("aisle")["reordered"].agg(['count',
temp_df = temp_df.sort_values('total', ascending = False).reset_index()
```

```
CPU times: user 1.34 s, sys: 240 ms, total: 1.58 s
Wall time: 1.71 s
```

In [103…
```
temp_df.head()
```

Out[103…

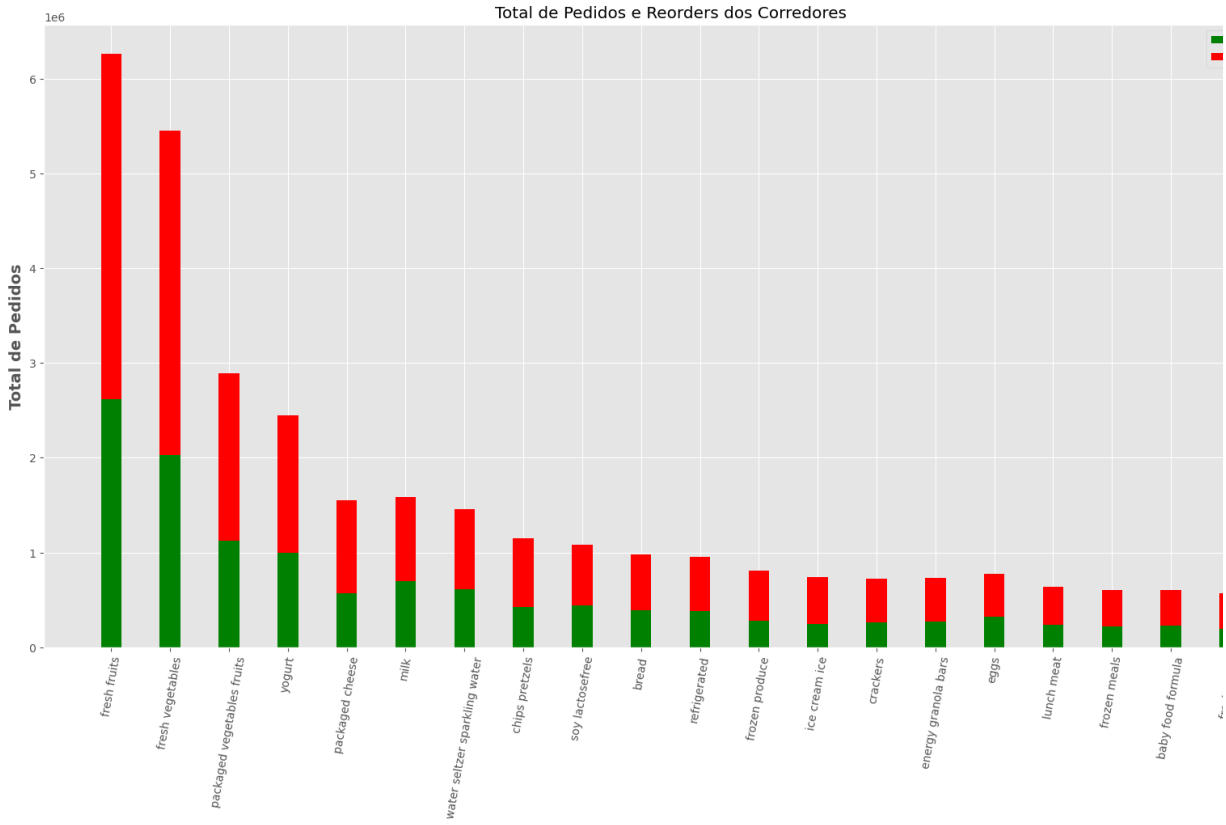
	aisle	total	reorders
0	fresh fruits	3642188	261546 9
1	fresh vegetables	3418021	203217 2
2	packaged vegetables fruits	1765317	1127177
3	yogurt	1452343	997018
4	packaged cheese	979763	573383

In [104…
```
# Listas
labels = list(temp_df.aisle[0:20])
reorder = list(temp_df.reorders[0:20])
total = list(temp_df.total[0:20])
```

In [105…
```
# Volume de ocorrencias e reorders nos principais corredores
temp_df = (
```

```
    order_product_prior.groupby('aisle')['reordered']
    .agg(total='count', reorders='sum')
    .sort_values('total', ascending=False)
    .head(20)
    .sort_values('total')
)

plt.figure(figsize=(10, 8))
plt.barh(temp_df.index, temp_df['total'], label='Total de ocorrencias')
plt.barh(temp_df.index, temp_df['reorders'], label='Reorders')
plt.title('Top 20 Corredores: Ocorrencias e Recompras', fontsize=15, font
plt.xlabel('Quantidade de Ocorrencias', fontsize=12)
plt.ylabel('Corredor', fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()
```



Relacao entre posicao de adicao ao carrinho e recompra

Vamos verificar se a posicao em que o produto e adicionado ao carrinho esta associada a proporcao de iter posteriormente classificados como `reordered`.

In [106…
```
# Suprime avisos nao essenciais durante a etapa exploratoria.
import warnings
warnings.filterwarnings("ignore")
```

In [107…
```
# Instalacao de `shutup` nao e necessaria para a analise e foi removida n
```

In [108…
```
# Uso de `shutup` removido na versao revisada.
```

In [109…

```
order_product_prior["add_to_cart_order_mod"] = order_product_prior["add_t
order_product_prior["add_to_cart_order_mod"].loc[order_product_prior["add
grouped_df = order_product_prior.groupby(["add_to_cart_order_mod"])["reor
```

In [110…

```
grouped_df.head(10)
```

Out[110…

	add_to_cart_order_mod	reordered
0	1	0.677533
1	2	0.676251
2	3	0.658037
3	4	0.636958
4	5	0.617383
5	6	0.600420
6	7	0.585687
7	8	0.573247
8	9	0.561474
9	10	0.551018

Implementacao do Algoritmo Apriori

O algoritmo Apriori sera aplicado sobre as transacoes formadas a partir dos pedidos anteriores.

Para manter o tempo de processamento adequado, os exemplos abaixo utilizam as `500.000` p transacoes da base `prior`. Portanto, as regras apresentadas representam essa amostra e nao necessariam a populacao de pedidos.

Metricas das regras de associacao

As principais metricas utilizadas neste projeto sao:

Support: proporcao de transacoes que contem simultaneamente os itens A e B.

Quanto maior o suporte, mais frequente e a combinacao na base analisada.

Confidence: proporcao de transacoes que contem B entre aquelas que contem A.

Em uma regra `A -> B`, a confianca indica a frequencia com que B aparece quando A esta presente.

Lift: mede quanto a ocorrencia de B e mais frequente na presenca de A em comparacao com a ocorrencia e de B isoladamente.

- `Lift > 1`: associacao positiva;
- `Lift = 1`: ausencia de associacao adicional;
- `Lift < 1`: associacao negativa.

In [111…
```
# Prepara a tupla com as transacoes
transactions_tup = [tuple(row) for row in transactions['Products'].tolist
```

Apriori - Support = 0,01 e Confidence = 0,20

Primeiro, executamos o algoritmo com suporte minimo de 1% e confianca minima de 20%.

In [112…
```
st = datetime.now()
itemsets_ap, rules_ap = apriori(transactions_tup[:500000], min_support =
print('Tempo de Execucao : ', datetime.now() - st)
```

```
Tempo de Execucao :  0:00:02.213223
```

In [113…
```
rules_ap
```

Out[113…
```
 [{21137} -> {13176},
  {21903} -> {13176},
  {27966} -> {13176},
  {47209} -> {13176},
  {16797} -> {24852},
  {21137} -> {24852},
  {27966} -> {21137},
  {21903} -> {24852},
  {28204} -> {24852},
  {47626} -> {24852},
  {47766} -> {24852},
  {49683} -> {24852}]
```

In [114…
```
# Listas de itens para considerar
item_A = [27966,47209,21137,47766,21903,49683,47626,28204,16797,21903,211
item_B = [13176,13176,24852,24852,24852,24852,24852,24852,24852,13176,131
temp = pd.DataFrame()
temp['itemA'] = item_A
temp['itemB'] = item_B
```

In [115…
```
# Listas para as metricas
support_A = []
support_B = []
support_AB = []
confidence_AB = []
lift_AB = []
```

In [116…
```
# Loop
for i in range(len(temp)):

    # Calcula o support de A
    support_A.append(itemsets_ap[1][tuple([temp['itemA'][i],])] / 500000)
```

```
    # Calcula o support de B
    support_B.append(itemsets_ap[1][tuple([temp['itemB'][i],])] / 500000)

    # Calcula o support de A e B
    if tuple([temp['itemA'][i],temp['itemB'][i]]) in itemsets_ap[2].keys(
        support_AB.append(itemsets_ap[2][tuple([temp['itemA'][i],temp['it
    else:
        support_AB.append(itemsets_ap[2][tuple([temp['itemB'][i],temp['it

    # Calcula a confidence
    confidence_AB.append(support_AB[i] / support_A[i])

    # Calcula o lift
    lift_AB.append(support_AB[i] / (support_A[i] * support_B[i]))
```

In [117…

```
# Dataframe com as regras de associacao
df_rules_ap = pd.DataFrame()
df_rules_ap['product_id'] = item_A
df_rules_ap = df_rules_ap.merge(product, on = 'product_id', how = 'left')
df_rules_ap['Product_A'] = df_rules_ap['product_name']
df_rules_ap = df_rules_ap.drop(columns = ['product_id','product_name','ai
df_rules_ap['product_id'] = item_B
df_rules_ap = df_rules_ap.merge(product, on = 'product_id', how = 'left')
df_rules_ap['Product_B'] = df_rules_ap['product_name']
df_rules_ap = df_rules_ap.drop(columns = ['product_id','product_name','ai
df_rules_ap['Support_A'] = support_A
df_rules_ap['Support_B'] = support_B
df_rules_ap['Support_AB'] = support_AB
df_rules_ap['Confidence_AB'] = confidence_AB
df_rules_ap['Lift_AB'] = lift_AB
```

In [118…

```
df_rules_ap.sort_values(by = 'Confidence_AB', ascending = False)
```

Out[118...

	Product_A	Product_B	Support_A	Support_B	Support_AB
7	Organic Fuji Apple	Banana	0.027942	0.147194	0.010630
5	Cucumber Kirby	Banana	0.030218	0.147194	0.010024
3	Organic Avocado	Banana	0.054564	0.147194	0.016328
0	Organic Raspberries	Bag of Organic Bananas	0.042334	0.117802	0.012428
1	Organic Hass Avocado	Bag of Organic Bananas	0.066218	0.117802	0.019260
8	Strawberries	Banana	0.044496	0.147194	0.012902
6	Large Lemon	Banana	0.047718	0.147194	0.012692
11	Organic Raspberries	Organic Strawberries	0.042334	0.082550	0.010524
10	Organic Strawberries	Bag of Organic Bananas	0.082550	0.117802	0.019122
4	Organic Baby Spinach	Banana	0.075000	0.147194	0.016016
2	Organic Strawberries	Banana	0.082550	0.147194	0.017596
9	Organic Baby Spinach	Bag of Organic Bananas	0.075000	0.117802	0.015500

In [119...

```
df_rules_ap.sort_values(by = 'Lift_AB', ascending = False)
```

Out[119…

	Product_A	Product_B	Support_A	Support_B	Support_AB
11	Organic Raspberries	Organic Strawberries	0.042334	0.082550	0.010524
7	Organic Fuji Apple	Banana	0.027942	0.147194	0.010630
0	Organic Raspberries	Bag of Organic Bananas	0.042334	0.117802	0.012428
1	Organic Hass Avocado	Bag of Organic Bananas	0.066218	0.117802	0.019260
5	Cucumber Kirby	Banana	0.030218	0.147194	0.010024
3	Organic Avocado	Banana	0.054564	0.147194	0.016328
8	Strawberries	Banana	0.044496	0.147194	0.012902
10	Organic Strawberries	Bag of Organic Bananas	0.082550	0.117802	0.019122
6	Large Lemon	Banana	0.047718	0.147194	0.012692
9	Organic Baby Spinach	Bag of Organic Bananas	0.075000	0.117802	0.015500
4	Organic Baby Spinach	Banana	0.075000	0.147194	0.016016
2	Organic Strawberries	Banana	0.082550	0.147194	0.017596

Apriori - Support = 0,005 e Confidence = 0,20

Em seguida, reduzimos o suporte minimo para 0,5%, mantendo a confianca minima em 20%. I permite identificar associacoes menos frequentes que podem apresentar alto potencial de relacionamento.

In [120…

```
st = datetime.now()
itemsets_ap_1, rules_ap_1 = apriori(transactions_tup[:500000], min_suppor
print('Tempo de Execucao: ', datetime.now() - st)
```

```
Tempo de Execucao:  0:00:04.041543
```

In [121…

```
rules_ap_1
```

Out[121…
```
[{4605} -> {24852},
 {4920} -> {24852},
 {5876} -> {13176},
 {5876} -> {47209},
 {8277} -> {13176},
 {8277} -> {24852},
 {9076} -> {24852},
 {19057} -> {13176},
 {21137} -> {13176},
 {21903} -> {13176},
 {22935} -> {13176},
 {27966} -> {13176},
 {30391} -> {13176},
 {37646} -> {13176},
 {39275} -> {13176},
 {45007} -> {13176},
 {47209} -> {13176},
 {16797} -> {24852},
 {21137} -> {24852},
 {27966} -> {21137},
 {30391} -> {21137},
 {39275} -> {21137},
 {21903} -> {24852},
 {24964} -> {22935},
 {26209} -> {24852},
 {27845} -> {24852},
 {28204} -> {24852},
 {30489} -> {24852},
 {37646} -> {24852},
 {40706} -> {24852},
 {42265} -> {24852},
 {44632} -> {24852},
 {45066} -> {24852},
 {47626} -> {24852},
 {47766} -> {24852},
 {49683} -> {24852},
 {31717} -> {26209},
 {30391} -> {47209}]
```

In [122…
```
# Lista de itens para considerar
item_A1 = [27966,47209,4605,21137,47766,21903,49683,5876,37646,40706,4762
           28204,27845,24964,45066,9076,16797,21903,8277,30391,21137,2796
           42265,30391,8277,4920,39275,44632]
item_B1 = [13176,13176,24852,24852,24852,24852,24852,47209,24852,24852,24
           24852,24852,22935,24852,24852,24852,13176,24852,47209,13176,21
           24852,21137,13176,24852,13176,24852]
temp1 = pd.DataFrame()
temp1['itemA'] = item_A1
temp1['itemB'] = item_B1
```

In [123…
```
# Listas de metricas
support_A1 = []
support_B1 = []
support_AB1 = []
confidence_AB1 = []
lift_AB1 = []
```

In [124…
```
# Loop
for i in range(len(temp1)):
```

```
    support_A1.append(itemsets_ap_1[1][tuple([temp1['itemA'][i],])] / 500

    support_B1.append(itemsets_ap_1[1][tuple([temp1['itemB'][i],])] / 500

    if tuple([temp1['itemA'][i],temp1['itemB'][i]]) in itemsets_ap_1[2].k
        support_AB1.append(itemsets_ap_1[2][tuple([temp1['itemA'][i],temp
    else:
        support_AB1.append(itemsets_ap_1[2][tuple([temp1['itemB'][i],temp

    confidence_AB1.append(support_AB1[i] / support_A1[i])

    lift_AB1.append(support_AB1[i] / (support_A1[i] * support_B1[i]))
```

In [125…

```
# Dataframe com as regras de associacao
df_rules_ap1 = pd.DataFrame()
df_rules_ap1['product_id'] = item_A1
df_rules_ap1 = df_rules_ap1.merge(product, on = 'product_id', how = 'left
df_rules_ap1['Product_A'] = df_rules_ap1['product_name']
df_rules_ap1 = df_rules_ap1.drop(columns = ['product_id','product_name','
df_rules_ap1['product_id'] = item_B1
df_rules_ap1 = df_rules_ap1.merge(product, on = 'product_id', how = 'left
df_rules_ap1['Product_B'] = df_rules_ap1['product_name']
df_rules_ap1 = df_rules_ap1.drop(columns = ['product_id','product_name','
df_rules_ap1['Support_A'] = support_A1
df_rules_ap1['Support_B'] = support_B1
df_rules_ap1['Support_AB'] = support_AB1
df_rules_ap1['Confidence_AB'] = confidence_AB1
df_rules_ap1['Lift_AB'] = lift_AB1
```

In [126…

```
df_rules_ap1.sort_values(by = 'Confidence_AB', ascending = False).head(10
```

Out[126…

	Product_A	Product_B	Support_A	Support_B	Support_AB	
16	Organic Fuji Apple	Banana	0.027942	0.147194	0.010630	0
19	Honeycrisp Apple	Banana	0.024774	0.147194	0.008832	0
6	Cucumber Kirby	Banana	0.030218	0.147194	0.010024	0
27	Organic Large Extra Fancy Fuji Apple	Bag of Organic Bananas	0.023374	0.117802	0.007330	0
4	Organic Avocado	Banana	0.054564	0.147194	0.016328	0
35	Seedless Red Grapes	Banana	0.026054	0.147194	0.007750	0
2	Yellow Onions	Banana	0.022814	0.147194	0.006750	0
0	Organic Raspberries	Bag of Organic Bananas	0.042334	0.117802	0.012428	0
20	Blueberries	Banana	0.017674	0.147194	0.005186	0
1	Organic Hass Avocado	Bag of Organic Bananas	0.066218	0.117802	0.019260	0

In [127…

```
df_rules_ap1.sort_values(by = 'Lift_AB', ascending = False).head(10)
```

Out[127...

	Product_A	Product_B	Support_A	Support_B	Support_AB
15	Organic Cilantro	Limes	0.021334	0.043640	0.005342
18	Organic Garlic	Organic Yellow Onion	0.034158	0.035306	0.006868
7	Organic Lemon	Organic Hass Avocado	0.027242	0.066218	0.006534
24	Organic Cucumber	Organic Hass Avocado	0.025056	0.066218	0.005396
26	Organic Raspberries	Organic Strawberries	0.042334	0.082550	0.010524
30	Organic Blueberries	Organic Strawberries	0.031188	0.082550	0.007284
27	Organic Large Extra Fancy Fuji Apple	Bag of Organic Bananas	0.023374	0.117802	0.007330
16	Organic Fuji Apple	Banana	0.027942	0.147194	0.010630
0	Organic Raspberries	Bag of Organic Bananas	0.042334	0.117802	0.012428
33	Organic Cucumber	Organic Strawberries	0.025056	0.082550	0.005138

Principais Insights das Regras de Associacao

Na configuracao com Support = 0,005 e Confidence = 0,20, algumas associacoes apresen elevado.

Entre os resultados exibidos no notebook:

Produto A	Produto B	Confidence	Lift
Organic Cilantro	Limes	25,04%	5,74
Organic Garlic	Organic Yellow Onion	20,11%	5,69
Organic Lemon	Organic Hass Avocado	23,99%	3,62
Organic Cucumber	Organic Hass Avocado	21,54%	3,25

Produto A	Produto B	Confidence	Lift
Organic Raspberries	Organic Strawberries	24,86%	3,01

Interpretacao

O Lift acima de 1 indica que a ocorrencia conjunta dos produtos e maior do que seria esperado con apenas a frequencia individual do segundo produto.

As associacoes de maior Lift podem ser utilizadas como ponto de partida para estrategias de cross-selling, recomendacao de produtos e composicao de ofertas. Entretanto, antes de uma decisao comercial, as regras ser avaliadas tambem quanto a volume, margem, disponibilidade e contexto do negocio.

Conclusao

A analise aplicou tecnicas de exploracao de dados e regras de associacao para identificar padroes de compr conjunto transacional.

A preparacao das cestas permitiu aplicar o algoritmo Apriori e avaliar associacoes por meio de Support, Co e Lift. Os resultados demonstram que alguns pares de produtos apresentam associacao significativamente s esperada pela frequencia individual.

Do ponto de vista de negocio, essas informacoes podem apoiar iniciativas de recomendacao, cross-selling e organizacao de ofertas. Para utilizacao em producao, seria necessario complementar a analise com criterios comerciais, como margem, estoque, preco e periodo de analise.

O projeto demonstra, portanto, a aplicacao pratica de Python, Pandas, analise exploratoria, preparacao de d regras de associacao em um problema de analise de dados.

Fim